



UNIVERSIDAD
DE
CÓRDOBA

GRADO EN INGENIERÍA INFORMÁTICA

Paralelización en Kotlin

Autores:

Antonio Raso Fernández i02rafea@uco.es

Docente:

Ezequiel Herruzo Gómez

Córdoba, 4 de junio de 2025

Índice

1. Introducción	2
1.1. Kotlin como lenguaje moderno	2
1.2. Por qué Kotlin es interesante para la paralelización: integración con la JVM, corrutinas y soporte multiplataforma	2
2. Corrutinas en Kotlin	2
2.1. Diferencias clave entre corrutinas e hilos:	2
2.2. Estructuras de las corrutinas	2
2.2.1. Scope: el contenedor del ciclo de vida	2
2.2.2. Builders	3
2.2.3. Dispatchers: control de asignación de hilos	3
2.3. Obtención de la paralelización	4
2.3.1. Secuencial	4
2.3.2. Concurrencia	5
2.3.3. Paralelización	5
3. Demostración de Paralelización de Archivos JSON en Kotlin	6
3.1. Modos de ejecución	6
4. Explicación del código	7
4.1. Lectura Secuencial	7
4.2. Paralelización con Dispatchers Personalizados	7
4.3. Paralelización con Dispatchers por Defecto	8
5. Análisis de Resultados	8
5.1. Características del Dispositivo de Prueba	8
5.2. Análisis de tiempos de ejecución	9
5.3. Cálculo del Speed-Up	11
6. Conclusiones	11
7. Repositorio Git-Hub	12
8. Bibliografía	12

1. Introducción

1.1. Kotlin como lenguaje moderno

Kotlin es un lenguaje de programación moderno, tipado estáticamente y de código abierto, desarrollado por JetBrains. Diseñado para ejecutarse en la Java Virtual Machine (JVM), también puede compilarse a JavaScript y código nativo, lo que lo hace versátil para diversas plataformas.

1.2. Por qué Kotlin es interesante para la paralelización: integración con la JVM, corrutinas y soporte multiplataforma

Kotlin ofrece características que lo hacen especialmente adecuado para la programación paralela y concurrente:

- **Corrutinas:** Kotlin introduce las corrutinas como una forma ligera y eficiente de manejar operaciones asíncronas y concurrentes. A diferencia de los hilos tradicionales, las corrutinas permiten escribir código asíncrono de manera secuencial, facilitando su comprensión y mantenimiento .
- **Soporte multiplataforma:** Con Kotlin Multiplatform, es posible compartir código común entre diferentes plataformas, como Android, iOS, backend y frontend web. Esto permite implementar soluciones concurrentes que funcionan de manera coherente en diversos entornos .

2. Corrutinas en Kotlin

A diferencia de los hilos tradicionales, las corrutinas son gestionadas por la biblioteca de Kotlin y no están directamente vinculadas a los hilos del sistema operativo.

2.1. Diferencias clave entre corrutinas e hilos:

1. **Gestión y peso:** Los hilos son gestionados por el sistema operativo. Por el contrario, las corrutinas son gestionadas por la biblioteca de Kotlin, permitiendo la creación de miles de ellas.
2. **Suspensión y reanudación:** Las corrutinas pueden suspender su ejecución sin bloquear el hilo, lo que permite que otros procesos utilicen ese hilo mientras la corrutina está suspendida. Los hilos, en cambio, cuando se suspenden, bloquean el hilo, impidiendo que se realicen otras tareas en él.
3. **Cambio de contexto:** Una corrutina puede iniciar su ejecución en un hilo y reanudarse en otro, ofreciendo una gran flexibilidad en la asignación de tareas. Los hilos, una vez iniciados, no cambian de contexto de ejecución de manera tan flexible.

2.2. Estructuras de las corrutinas

2.2.1. Scope: el contenedor del ciclo de vida

Define el contexto en el que se ejecutarán las corrutinas

- **GlobalScope:** se ejecuta durante toda la vida de la aplicación.
- **CoroutineScope:** creado explícitamente y ligado al ciclo de vida del componente.

```
1 fun main() {  
2     // Nivel 1: Declaración de un Scope  
3     val scope = CoroutineScope()  
4 }
```

2.2.2. Builders

Dentro del scope, usas builders para lanzar corrutinas:

- **launch**: lanza una corrutina que no devuelve valor.
- **async**: lanza una corrutina que devuelve un resultado.
- **withContext**: cambia de contexto dentro de una corrutina ya existente.

```
1 fun main() {
2     val scope = CoroutineScope()
3
4     scope.launch {
5         // Nivel 2: Builder 'launch'
6     }
7
8     scope.async {
9         // Nivel 2: Builder 'async'
10    }
11
12    scope.launch {
13        withContext() {
14            // Nivel 2: Cambio de contexto
15        }
16    }
17 }
```

2.2.3. Dispatchers: control de asignación de hilos

Las corrutinas de Kotlin utilizan despachadores para determinar qué subprocesos se utilizan para la ejecución de corrutinas.

- **Default**: Este despachador está optimizado para realizar trabajo que usa la CPU de manera intensiva fuera del subproceso principal, el cual utiliza los hilos permitidos por la CPU (multinúcleo).
- **IO**: Este despachador está optimizado para realizar E/S de disco o red fuera del subproceso principal.
- **Main**: Utiliza este despachador para ejecutar una corrutina en el subproceso de Android principal.

```
1 fun main() {
2     val scope = CoroutineScope(Dispatchers.Default)
3
4     scope.launch {
5         // Nivel 2: Builder 'launch'
6     }
7
8     scope.async {
9         // Nivel 2: Builder 'async'
10    }
11
12    scope.launch {
13        withContext(Dispatchers.IO) {
14            // Nivel 2: Cambio de contexto
15        }
16    }
17 }
```

2.3. Obtención de la paralelización

A continuación, mostraré un ejemplo sobre cómo obtener la paralelización de forma sencilla.

2.3.1. Secuencial

```
1 fun tarea1() {
2     println("Inicio tarea1")
3     Thread.sleep(1000)
4     println("Fin tarea1")
5 }
6
7 fun subTarea2A() {
8     println("Inicio subTarea2A")
9     Thread.sleep(1000)
10    println("Fin subTarea2A")
11 }
12
13 fun subTarea2B() {
14     println("Inicio subTarea2B")
15     Thread.sleep(700)
16     println("Fin subTarea2B")
17 }
18
19 fun tarea2() {
20     println("Inicio tarea2")
21     subTarea2A()
22     subTarea2B()
23     println("Fin tarea2")
24 }
25
26 fun main() {
27     val start = System.currentTimeMillis()
28     tarea1()
29     tarea2()
30     val end = System.currentTimeMillis()
31     println("Tiempo total: ${end - start} ms")
32 }
```

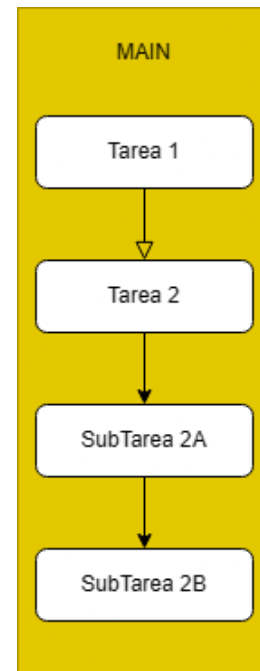


Figura 1: Diagrama de flujo

De forma secuencial el código tiene una duración aproximada de *2700ms*

2.3.2. Concurrencia

```

1 fun main() {
2     runBlocking {
3         val start = System.currentTimeMillis()
4
5         val job1 = launch { tarea1() }
6         val job2 = launch { tarea2() }
7
8         job1.join()
9         job2.join()
10
11         val end = System.currentTimeMillis()
12         println("Tiempo total: ${end - start} ms")
13     }
14 }

```

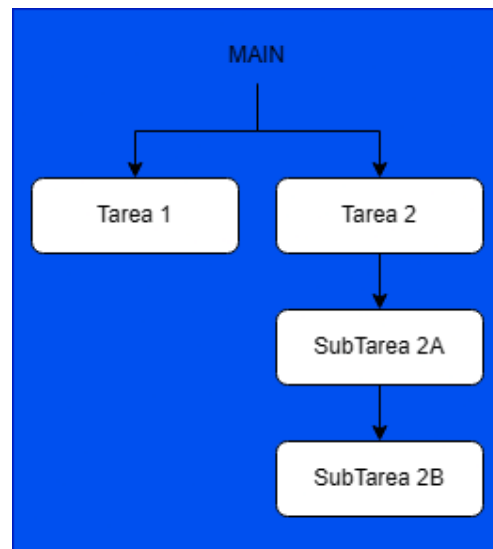


Figura 2: Diagrama de flujo

De forma concurrente el código tiene una duración aproximada de *1700ms*

2.3.3. Paralelización

```

1 suspend fun tarea2() = coroutineScope {
2     println("Inicio tarea2")
3     val jobA = async{ subTarea2A() }
4     val jobB = async{ subTarea2B() }
5     jobA.await()
6     jobB.await()
7     println("Fin tarea2")
8 }
9
10 fun main() {
11     runBlocking {
12         val start = System.currentTimeMillis()
13         val job1 = async(Dispatchers.Default) {
14             tarea1()
15         }
16         val job2 = async(Dispatchers.Default) {
17             tarea2()
18         }
19         job1.await()
20         job2.await()
21         val end = System.currentTimeMillis()
22         println("Tiempo total: ${end - start} ms")
23     }
24 }

```

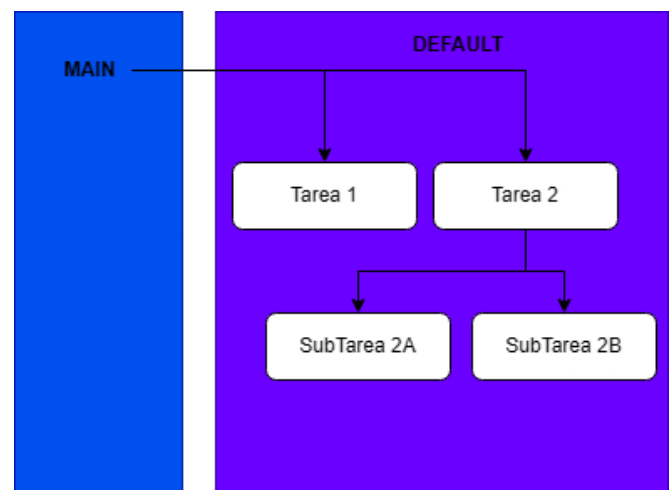


Figura 3: Diagrama de flujo paralelo

De forma paralela, el código tiene una duración aproximada de *1000ms*

3. Demostración de Paralelización de Archivos JSON en Kotlin

El procesamiento de múltiples archivos JSON de forma secuencial puede convertirse en un cuello de botella significativo en aplicaciones que requieren velocidad y escalabilidad. Cada archivo debe ser leído desde disco (una operación I/O relativamente lenta) y posteriormente interpretado o parseado en estructuras de datos utilizables dentro del programa.

Este proyecto aborda dicho problema mediante la paralelización del proceso de lectura y parseo de archivos JSON. Se emplea un enfoque basado en corrutinas de Kotlin que permite realizar estas tareas de manera concurrente: se utilizan dispatchers `IO` para realizar la lectura simultánea de los archivos desde el sistema de archivos, y dispatchers `Default` para llevar a cabo el parseo en memoria de los datos a objetos Kotlin.

Cada archivo JSON contiene información estructurada similar al siguiente ejemplo:

```
{
  "name": "Adeel Solangi",
  "language": "Sindhi",
  "id": "V590F92YF627HIFY0",
  "bio": "Donec lobortis eleifend condimentum...",
  "version": 6.1
}
```

Estos datos se mapean a una clase de datos en Kotlin denominada `Datos`, definida como:

```
data class Datos(
    @SerializedName("name") val first_name: String,
    @SerializedName("language") val language: String,
    @SerializedName("id") val id: String,
    @SerializedName("bio") val bio: String,
    @SerializedName("version") val version: Float,
)
```

3.1. Modos de ejecución

El sistema implementado permite evaluar el rendimiento de tres enfoques diferentes para la lectura y parseo de archivos JSON, configurables mediante la interfaz de usuario:

1. **Modo secuencial:** En este modo, tanto la lectura de los archivos JSON como su parseo a objetos Kotlin se realiza de forma completamente secuencial, uno tras otro, sin paralelismo. Es útil como línea base para comparar el rendimiento con respecto a los otros dos modos. Este comportamiento se activa al marcar la opción “Secuencial” en la interfaz gráfica.
2. **Modo paralelo personalizado:** En este enfoque se emplean corrutinas con despachadores limitados en cuanto a número de hilos. El usuario puede ajustar el número de hilos efectivos mediante un control deslizante.
 - La lectura de los archivos desde disco se realiza en un conjunto de corrutinas asociadas a un `Dispatcher.IO.limitedParallelism(n)`, donde `n` es el número de hilos elegido.
 - El parseo de los textos JSON a listas de objetos `Datos` se realiza en paralelo utilizando `Dispatcher.Default.limitedParallelism(n)`.

Este modo permite controlar el grado de concurrencia y observar el impacto del paralelismo en el rendimiento global.

3. **Modo por defecto en Kotlin:** En este caso se recurre a los despachadores por defecto de Kotlin sin limitar explícitamente el número de hilos. Tanto las tareas de I/O como de parseo usan `Dispatchers.IO` y `Dispatchers.Default`, respectivamente. Esto permite aprovechar al máximo la gestión de hilos del sistema y el balanceo de carga que realiza Kotlin internamente. Se activa seleccionando la opción “Dispatchers Kotlin”.

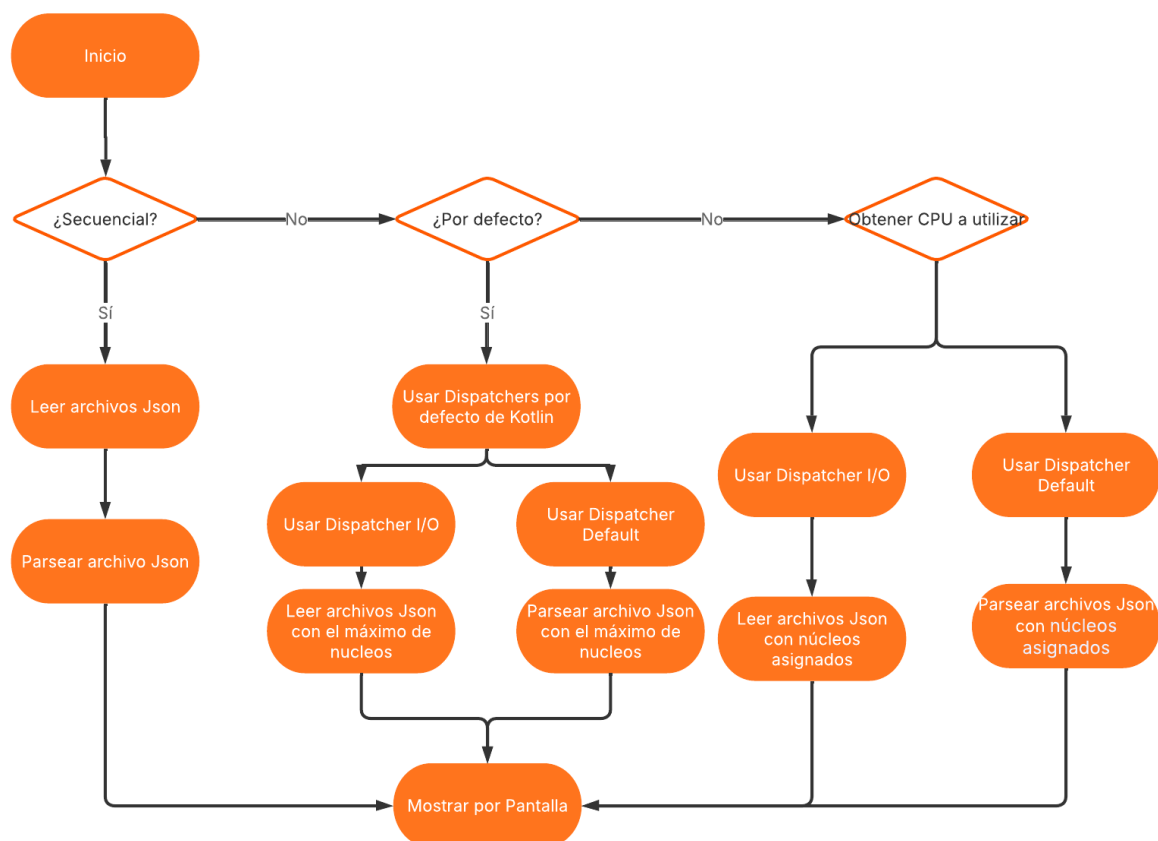


Figura 4: Diagrama de flujo

4. Explicación del código

A continuación se presentan los fragmentos más representativos del código que ilustran la implementación de los tres modos de procesamiento de archivos JSON: secuencial, paralelo personalizado y usando `Dispatchers.Default`.

4.1. Lectura Secuencial

La lectura secuencial se realiza recorriendo todos los archivos uno por uno y decodificando su contenido sin uso de corrutinas asíncronas ni dispatchers personalizados.

```

datosProcesados = (1..cantidadArchivosJson).flatMap {
    val contenidoJson = leerJson(contexto, it % 5)
    json.decodeFromString<List<Datos>>(contenidoJson)
}
  
```

4.2. Paralelización con Dispatchers Personalizados

En este modo se utilizan:

- `Dispatchers.IO.limitedParallelism(n)` para la lectura
- `Dispatchers.Default.limitedParallelism(n)` para el parseo

permitiendo controlar el número de hilos implicados.

```
val dispatcherParser = Dispatchers.Default.limitedParallelism(hilosEfectivos)
val dispatcherJson = Dispatchers.IO.limitedParallelism(hilosEfectivos)

val contenidosJson = (1..cantidadArchivosJson).map {
    async(dispatcherJson) {
        leerJson(contexto, it % 5)
    }
}

val listasJson = contenidosJson.map { contenidoJson ->
    async(dispatcherParser) {
        json.decodeFromString<List<Datos>>>(contenidoJson.await())
    }
}

datosProcesados = listasJson.awaitAll().flatten()
```

4.3. Paralelización con Dispatchers por Defecto

Cuando se marca la opción de usar Dispatchers por defecto, sin limitar el paralelismo manualmente.

- **Dispatchers.IO** para la lectura
- **Dispatchers.Default**) para el parseo

```
val dispatcherParser = Dispatchers.Default
val dispatcherJson = Dispatchers.IO
```

Esta forma de ejecución puede ofrecer buen rendimiento en muchos casos, aunque se pierde el control fino sobre el número de hilos simultáneos.

5. Análisis de Resultados

Para evaluar el rendimiento de los distintos modos de procesamiento (secuencial, paralelo personalizado y usando **Dispatchers.Default**), se ha utilizado como dispositivo de pruebas un **OnePlus 11**, cuyas especificaciones técnicas permiten una ejecución representativa en entornos móviles de gama alta.

5.1. Características del Dispositivo de Prueba

El experimento se ha realizado en un dispositivo **OnePlus 11**, con las siguientes especificaciones clave:

- **Procesador:** Qualcomm Snapdragon[®] 8 Gen 2 (4 nm), CPU de ocho núcleos (1x3.2 GHz Cortex-X3, 2x2.8 GHz Cortex-A715, 2x2.8 GHz Cortex-A710, 3x2.0 GHz Cortex-A510)

Organización de la memoria caché:

- **Caché L1 (por núcleo):**
 - Cortex-X3: 64KB de instrucciones + 64KB de datos.
 - Cortex-A715 / Cortex-A710: 64KB de instrucciones + 64KB de datos por núcleo.
 - Cortex-A510: 64KB de instrucciones + 64KB de datos por núcleo.

■ **Caché L2:**

- Cortex-X3: 1 MB.
- Cortex-A715 / Cortex-A710: 512 KB por núcleo.
- Cortex-A510: 128 KB por núcleo.

■ **Caché L3 (compartida): 8 MB.**

5.2. Análisis de tiempos de ejecución

	Iteración 1	Iteración 2	Iteración 3	Iteración 4	Media
Secuencial	201 ms	209 ms	219 ms	227 ms	214 ms
Default	236 ms	230 ms	255 ms	281 ms	250,5 ms
2 Núcleos	208 ms	218 ms	310 ms	245 ms	245,25 ms
4 Núcleos	226 ms	237 ms	243 ms	247 ms	238,25 ms
6 Núcleos	213 ms	233 ms	221 ms	215 ms	220,5 ms
8 Núcleos	207 ms	211 ms	221 ms	218 ms	214,25 ms

Tabla 1: Resultados para 1 archivo JSON

	Iteración 1	Iteración 2	Iteración 3	Iteración 4	Media
Secuencial	410 ms	476 ms	493 ms	426 ms	451,25 ms
Default	261 ms	267 ms	269 ms	273 ms	267,5 ms
2 Núcleos	284 ms	308 ms	287 ms	307 ms	296,5 ms
4 Núcleos	279 ms	292 ms	301 ms	292 ms	291 ms
6 Núcleos	262 ms	304 ms	286 ms	290 ms	285,5 ms
8 Núcleos	283 ms	248 ms	271 ms	300 ms	275,5 ms

Tabla 2: Resultados para 2 archivos JSON

	Iteración 1	Iteración 2	Iteración 3	Iteración 4	Media
Secuencial	924 ms	878 ms	977 ms	846 ms	906,25 ms
Default	391 ms	377 ms	342 ms	389 ms	374,75 ms
2 Núcleos	595 ms	490 ms	489 ms	520 ms	523,5 ms
4 Núcleos	384 ms	422 ms	409 ms	402 ms	404,25 ms
6 Núcleos	401 ms	443 ms	412 ms	421 ms	419,25 ms
8 Núcleos	387 ms	401 ms	403 ms	422 ms	403,25 ms

Tabla 3: Resultados para 4 archivos JSON

	Iteración 1	Iteración 2	Iteración 3	Iteración 4	Media
Secuencial	1821 ms	1789 ms	1780 ms	1834 ms	1806 ms
Default	816 ms	831 ms	888 ms	798 ms	833,25 ms
2 Núcleos	1199 ms	1138 ms	1153 ms	1123 ms	1153,25 ms
4 Núcleos	790 ms	804 ms	816 ms	886 ms	824 ms
6 Núcleos	832 ms	861 ms	856 ms	836 ms	846,25 ms
8 Núcleos	787 ms	849 ms	786 ms	840 ms	815,5 ms

Tabla 4: Resultados para 8 archivos JSON

	Iteración 1	Iteración 2	Iteración 3	Iteración 4	Media
Secuencial	4350 ms	4746 ms	5968 ms	5747 ms	5202,75 ms
Default	2132 ms	2181 ms	2160 ms	2078 ms	2137,75 ms
2 Núcleos	2665 ms	2676 ms	2882 ms	2678 ms	2725,25 ms
4 Núcleos	2054 ms	2013 ms	2032 ms	2024 ms	2030,75 ms
6 Núcleos	2110 ms	2077 ms	2108 ms	2045 ms	2085 ms
8 Núcleos	2112 ms	2051 ms	2115 ms	2120 ms	2099,5 ms

Tabla 5: Resultados para 20 archivos JSON

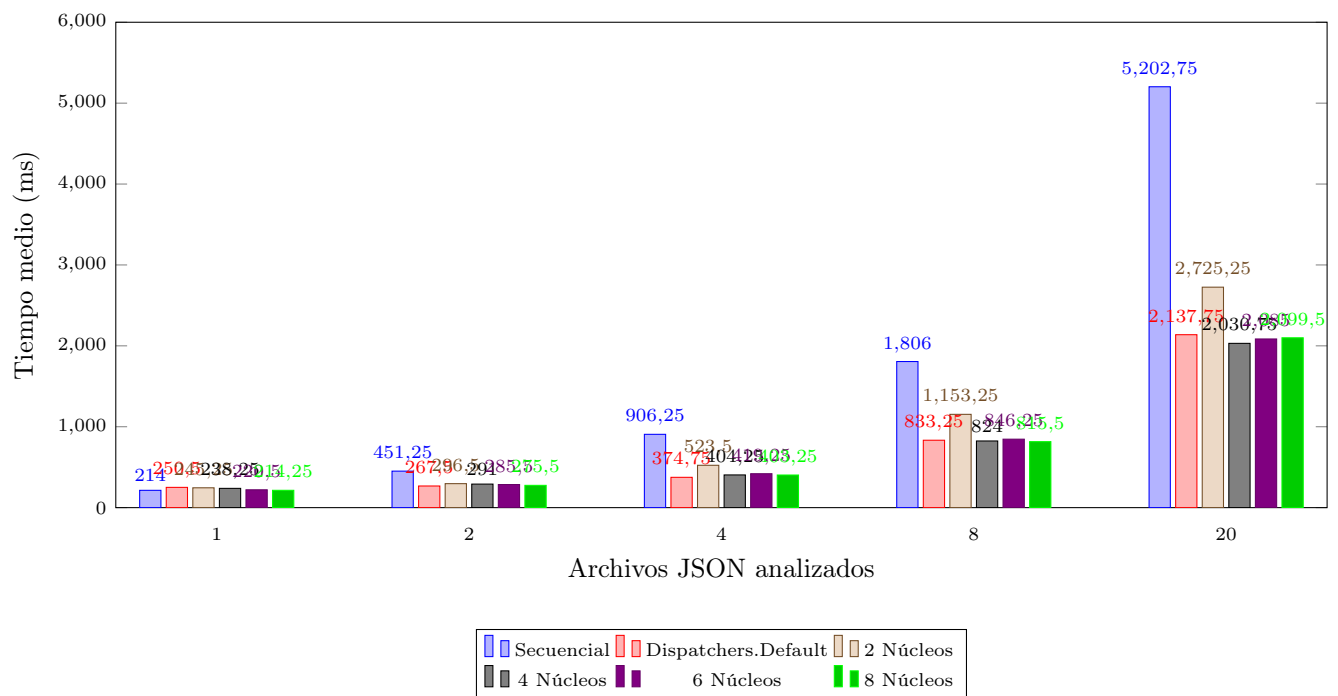


Figura 5: Rendimiento medio por número de archivos JSON analizados

5.3. Cálculo del Speed-Up

Porcentaje de mejora basados en speedup:

Archivos JSON	Default	2 Hilos	4 Hilos	6 Hilos	8 Hilos
1 archivo	85.00 %	87.00 %	90.00 %	97.00 %	100.00 %
2 archivos	169.00 %	152.00 %	155.00 %	158.00 %	164.00 %
4 archivos	242.00 %	173.00 %	224.00 %	216.00 %	225.00 %
8 archivos	217.00 %	157.00 %	219.00 %	213.00 %	222.00 %
20 archivos	243.00 %	191.00 %	256.00 %	249.00 %	248.00 %

Tabla 6: Porcentaje de mejora basado en el speedup para cada número de hilos

Los tiempos de ejecución muestran una evolución clara del rendimiento según el número de archivos JSON analizados y el modo de paralelización utilizado.

- **Ejecución secuencial:** Este modo presenta el mayor tiempo medio de ejecución para todos los casos. A medida que se incrementa el número de archivos, los tiempos escalan linealmente, alcanzando más de 5 segundos con 20 archivos.
- **Uso de Dispatchers.Default:** Esta opción ofrece una mejora sustancial en comparación con la ejecución secuencial, especialmente con cargas grandes.
- **Paralelización con núcleos fijos:** La configuración con **4 núcleos** proporciona un equilibrio notable entre velocidad y eficiencia.
- **Escalabilidad:** A partir de 6 núcleos, los beneficios adicionales empiezan a ser marginales. De hecho, con 8 núcleos, los tiempos son ligeramente superiores a los de 4 núcleos.
- **Rendimientos decrecientes con más hilos:** Aunque el uso de más hilos incrementa el rendimiento, este crecimiento no es lineal. A partir de cierto punto (por ejemplo, entre 6 y 8 hilos), las mejoras se estabilizan debido a factores como la sobrecarga en la gestión de hilos, la competencia por recursos y las limitaciones físicas del hardware.

6. Conclusiones

A partir de los resultados obtenidos, se extraen las siguientes conclusiones:

- **Sencillez de implementación:** La utilización de corrutinas en Kotlin ha permitido una implementación paralela clara, expresiva y relativamente sencilla, sin necesidad de gestionar manualmente hilos o sincronización compleja.
- **Expectativas frente a resultados:** Aunque se observaron mejoras notables con la paralelización, especialmente con grandes volúmenes de archivos JSON, se esperaba una mejora más significativa en los tiempos de ejecución para casos intermedios como la lectura y parseo de 5 archivos. En esos escenarios, el rendimiento paralelo no se aproximó tanto como se esperaba al tiempo de ejecución del caso con 1 archivo.
- **Consideración de alternativas:** En contextos donde el máximo rendimiento es crítico, como procesamiento intensivo o en tiempo real, podría ser más eficiente utilizar enfoques alternativos como la paralelización por defecto de Java (por ejemplo, `ForkJoinPool` o `parallelStream`) o bibliotecas específicas como OpenMP en entornos nativos. Estas tecnologías podrían ofrecer una gestión más afinada del paralelismo y un aprovechamiento más efectivo del hardware.

7. Repositorio Git-Hub

El código realizado en este proyecto está en el siguiente repositorio de Git-hub:

- Paralelizar Json en Kotlin

8. Bibliografía

- IO and Default Dispatchers in Kotlin Coroutines
- coroutines-overview
- Corrutinas de Kotlin en Android
- Introducción a las corrutinas en el Playground de Kotlin
- Curso Android & Kotlin
- Understanding Kotlin Coroutines
- KotlinSerialization
- Coroutine vs. Thread
- Understanding Kotlin's coroutines
- Especificaciones OnePlus 11